

LAPORAN TUGAS 6

Mata Kuliah Sistem Operasi



Di Susun Oleh :
Muhammad Rafi Amiruddin
Nim (2410651027)

**Jurusan Teknik Informatika
Fakultas Teknik
Universitas Muhammadiyah Jember
2025**

DAFTAR ISI

LAPORAN TUGAS 6.....	1
DAFTAR ISI	i
1 ISI LAPORAN	1
1.1 Tugas 1.....	1
1.1.1 Analisis Deadlock	1
1.2 Tugas 2.....	2
1.2.1 Implementasi Pencegahan	2
1.2.2 Teknik Try-Lock.....	5
1.2.3 Teknik Try-Lock.....	7
1.3 Tugas 3.....	8
1.3.1 Banker's Algorithm	8
1.4 Tugas 4.....	12
1.4.1 Kasus Nyata (Real Case).....	12

1 ISI LAPORAN

1.1 Tugas 1

1.1.1 Analisis Deadlock

- Jalankan program `deadlock_simple.c` dan jelaskan

1. Mengapa program mengalami deadlock?
2. Kondisi deadlock apa saja yang terpenuhi?
3. Gambar diagram resource allocation graph-nya

Jawab :

1. Mengapa program mengalami deadlock?

Program ini deadlock karena Thread 1 dan Thread 2 saling menunggu kunci (mutex) yang sedang dipegang oleh satu sama lain.

Urutan kejadiannya:

Thread 1 ngambil Lock 1, lalu mau ngambil Lock 2

Thread 2 ngambil Lock 2, lalu mau ngambil Lock 1

Akhirnya:

Thread 1 nunggu Lock 2 (yang lagi dipegang Thread 2)

Thread 2 nunggu Lock 1 (yang lagi dipegang Thread 1)

Karena dua-duanya nunggu, ya udah... program bakal nge-freeze selamanya deadlock.

2. Kondisi deadlock apa saja yang terpenuhi?

Ada 4 syarat deadlock (Coffman conditions). Keempat-empatnya terjadi di program ini.

1. Mutual Exclusion (Saling Mengunci)

Mutex cuma bisa dipegang satu thread pada satu waktu

2. Hold and Wait (Memegang & Menunggu)

Thread 1 sudah memegang Lock 1 dan menunggu Lock 2

Thread 2 sudah memegang Lock 2 dan menunggu Lock 1

3. No Preemption (Tidak Bisa Direbut Paksa)

Mutex tidak bisa direbut paksa. Harus dilepas oleh thread pemegang.

4. Circular Wait (Menunggu Melingkar)

Urutan menunggu membentuk lingkaran:

Thread 1 → butuh Lock 2

Thread 2 → butuh Lock 1

3. Diagram Resource Allocation Graph (RAG)

Thread 1 ---memegang---> Lock 1

Thread 1 ---menunggu---> Lock 2

Thread 2 ---memegang---> Lock 2

Thread 2 ---menunggu---> Lock 1

1.2 Tugas 2

1.2.1 Implementasi Pencegahan

- Modifikasi program deadlock_simple.c menggunakan teknik:

1. Lock ordering
2. Try-lock dengan retry
3. Bandingkan kedua metode tersebut

- Buat file nano Lock_Ordering.c

Yang didalam nya berisi syntax :

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
```

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
```

```
// Thread 1
```

```
void* thread1_func(void* arg) {
    printf("Thread 1: Mencoba mengambil Lock 1...\n");
    pthread_mutex_lock(&lock1);
    printf("Thread 1: Dapat Lock 1\n");

    sleep(1);
```

```

    printf("Thread 1: Mencoba mengambil Lock 2...\n");
    pthread_mutex_lock(&lock2);
    printf("Thread 1: Dapat Lock 2\n");

    printf("Thread 1: Selesai bekerja\n");

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return NULL;
}

// Thread 2 — urutannya dipaksa sama: lock1 -> lock2
void* thread2_func(void* arg) {
    printf("Thread 2: Mencoba mengambil Lock 1...\n");
    pthread_mutex_lock(&lock1);
    printf("Thread 2: Dapat Lock 1\n");

    sleep(1);

    printf("Thread 2: Mencoba mengambil Lock 2...\n");
    pthread_mutex_lock(&lock2);
    printf("Thread 2: Dapat Lock 2\n");

    printf("Thread 2: Selesai bekerja\n");

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return NULL;
}

int main() {
    pthread_t t1, t2;
    printf("=== LOCK ORDERING ===\n\n");

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\nProgram selesai tanpa deadlock!\n");
    return 0;
}

```

- Lalu Compile dan jalankan menggunakan gcc Lock_Ordering.c -o Lock_Ordering -lpthread dan ./lock_Ordering

Program ini nunjukkin cara mencegah deadlock dengan cara maksa semua thread ngambil lock dalam urutan yang sama.

Biasanya deadlock terjadi kalau:

Thread A ngambil Lock 1 dulu, terus nunggu Lock 2

Thread B ngambil Lock 2 dulu, terus nunggu Lock 1
akhirnya dua-duanya nunggu selamanya.

Nah, di program ini:

Semua thread HARUS ngambil Lock 1 dulu

Baru boleh ngambil Lock 2
Urutannya selalu sama, jadi nggak ada tabrakan.

Makanya output-nya keliatan:

Thread 2 ambil Lock 1 → Lock 2 → kerja → selesai

Thread 1 ambil Lock 1 → Lock 2 → kerja → selesai

Karena nge-lock-nya rapi dan berurutan, sistem nggak pernah ketuker urutan lock, jadi:

```

rafi@satelit: ~
rafi@satelit:~$ nano Lock_Ordering.c
rafi@satelit:~$ gcc Lock_Ordering.c -o Lock_Ordering -lpthread
rafi@satelit:~$ ./Lock_Ordering
=== LOCK ORDERING ===

Thread 2: Mencoba mengambil Lock 1...
Thread 2: Dapat Lock 1
Thread 1: Mencoba mengambil Lock 1...
Thread 2: Mencoba mengambil Lock 2...
Thread 2: Dapat Lock 2
Thread 2: Selesai bekerja
Thread 1: Dapat Lock 1
Thread 1: Mencoba mengambil Lock 2...
Thread 1: Dapat Lock 2
Thread 1: Selesai bekerja

Program selesai tanpa deadlock!

```

1.2.2 Teknik Try-Lock

- Buat file nano TryLock.c

Yang didalam nya berisi syntax :

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
```

```
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;
```

```
void* thread1_func(void* arg) {
    while (1) {
        printf("Thread 1: Coba Lock 1...\n");
        pthread_mutex_lock(&lock1);

        sleep(1); // simulasi kerja

        printf("Thread 1: Coba Lock 2...\n");
        if (pthread_mutex_trylock(&lock2) == 0) {
            printf("Thread 1: Dapat kedua lock!\n");
            break;
        } else {
            printf("Thread 1: Gagal dapat Lock 2, retry...\n");
            pthread_mutex_unlock(&lock1);
            usleep(100000); // kecil supaya tidak ketemu livelock
        }
    }

    printf("Thread 1: Selesai bekerja\n");
    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return NULL;
}
```

```
void* thread2_func(void* arg) {
    while (1) {
        printf("Thread 2: Coba Lock 2...\n");
        pthread_mutex_lock(&lock2);

        sleep(1);

        printf("Thread 2: Coba Lock 1...\n");
        if (pthread_mutex_trylock(&lock1) == 0) {
            printf("Thread 2: Dapat kedua lock!\n");
            break;
        } else {
            printf("Thread 2: Gagal dapat Lock 1, retry...\n");
        }
    }
}
```

```

        pthread_mutex_unlock(&lock2);
        usleep(100000);
    }
}

printf("Thread 2: Selesai bekerja\n");
pthread_mutex_unlock(&lock1);
pthread_mutex_unlock(&lock2);
return NULL;
}

int main() {
    pthread_t t1, t2;
    printf("=== TRY-LOCK + RETRY ===\n\n");

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\nProgram selesai tanpa deadlock!\n");
    return 0;
}

```

- Lalu Compile dan jalankan menggunakan gcc TryLock.c -o TryLock -lpthread dan ./TryLock

Penjelasan singkat, bahasa tidak baku

Dua thread butuh dua lock (Lock 1 dan Lock 2).

Tapi mereka nggak nunggu kalau lock kedua lagi dipake.

Mereka coba ambil lock pakai pthread_mutex_trylock().

Kalau gagal dapet lock kedua → langsung dilepas yang pertama, terus coba ulang dari awal.

Karena mereka nggak paksa nunggu dan selalu restart kalau gagal, akhirnya nggak ada situasi saling tunggu yang bikin deadlock.

Hasilnya:

Thread 1 gagal dapet Lock 2 → dia restart.

Thread 2 kebagian kedua lock duluan → selesai.

Thread 1 coba lagi → akhirnya dapat juga.

Program selesai aman tanpa deadlock.

Makna utamanya

Trylock + retry = kalau lock rebutan, jangan nunggu, tapi mundur dulu dan coba lagi.

Cara ini efektif buat ngindarin deadlock tanpa harus maksa urutan lock yang ketat.

```
rafi@satelit:~$ nano TryLock.c
rafi@satelit:~$ gcc TryLock.c -o TryLock -lpthread
rafi@satelit:~$ ./TryLock
=== TRY-LOCK + RETRY ===

Thread 1: Coba Lock 1...
Thread 2: Coba Lock 2...
Thread 1: Coba Lock 2...
Thread 1: Gagal dapat Lock 2, retry...
Thread 2: Coba Lock 1...
Thread 2: Dapat kedua lock!
Thread 2: Selesai bekerja
Thread 1: Coba Lock 1...
Thread 1: Coba Lock 2...
Thread 1: Dapat kedua lock!
Thread 1: Selesai bekerja

Program selesai tanpa deadlock!
```

1.2.3 Teknik Try-Lock

Metode	Deadlock?	Livelock?	Kecepatan	Kompleksitas	Kapan dipakai
Lock Ordering	Tidak	Jarang	Cepat	Simpel	Saat semua lock dapat diurutkan
Try-Lock + Retry	Tidak	Mungkin	Lebih lambat	Lebih rumit	Saat urutan lock tidak bisa ditentukan

1.3 Tugas 3

1.3.1 Banker's Algorithm

- Buat program baru dengan data:

4 proses

3 jenis resource

Available: [5, 4, 3]

Implementasikan request resource dan cek keamanan system

- Buat File nano Bankers_Algoritma.c

File tersebut berisi syntax :

```
#include <stdio.h>
```

```
#define P 4 // jumlah proses
```

```
#define R 3 // jumlah resource
```

```
int available[R] = {5, 4, 3};
```

```
int max[P][R] = {  
    {7, 5, 3},  
    {3, 2, 2},  
    {9, 0, 2},  
    {2, 2, 2}  
};
```

```
int allocation[P][R] = {  
    {0, 1, 0},  
    {2, 0, 0},  
    {3, 0, 2},  
    {2, 1, 1}  
};
```

```
int need[P][R];
```

```
// Hitung need = max - allocation
```

```
void calculate_need() {  
    for (int i = 0; i < P; i++) {  
        for (int j = 0; j < R; j++) {  
            need[i][j] = max[i][j] - allocation[i][j];  
        }  
    }  
}
```

```
int is_safe() {  
    int work[R];  
    int finish[P] = {0};
```

```

    for (int i = 0; i < R; i++)
        work[i] = available[i];

    int count = 0;

    while (count < P) {
        int found = 0;
        for (int i = 0; i < P; i++) {
            if (!finish[i]) {
                int can_run = 1;
                for (int j = 0; j < R; j++) {
                    if (need[i][j] > work[j]) {
                        can_run = 0;
                        break;
                    }
                }

                if (can_run) {
                    for (int j = 0; j < R; j++)
                        work[j] += allocation[i][j];
                    finish[i] = 1;
                    found = 1;
                    count++;
                }
            }
        }

        if (!found)
            return 0; // unsafe
    }

    return 1; // safe
}

int main() {
    calculate_need();

    int p;
    int req[R];

    printf("=== BANKER'S ALGORITHM ===\n");
    printf("Proses yang meminta resource (0-3): ");
    scanf("%d", &p);

    printf("Masukkan request resource (3 angka, format: A B C): ");
    for (int i = 0; i < R; i++)
        scanf("%d", &req[i]);

    // Cek apakah request ≤ need
    for (int i = 0; i < R; i++) {

```

```

        if (req[i] > need[p][i]) {
            printf("ERROR: Request melebihi kebutuhan proses!\n");
            return 0;
        }
    }

    // Cek apakah request ≤ available
    for (int i = 0; i < R; i++) {
        if (req[i] > available[i]) {
            printf("Resource tidak cukup, proses harus WAIT.\n");
            return 0;
        }
    }

    // Simulasi alokasi sementara
    for (int i = 0; i < R; i++) {
        available[i] -= req[i];
        allocation[p][i] += req[i];
        need[p][i] -= req[i];
    }

    if (is_safe()) {
        printf("STATUS: Sistem dalam keadaan SAFE. Request
DISETUIJUI.\n");
    } else {
        printf("STATUS: UNSAFE! Request DITOLAK dan state
dikembalikan.\n");

        // rollback
        for (int i = 0; i < R; i++) {
            available[i] += req[i];
            allocation[p][i] -= req[i];
            need[p][i] += req[i];
        }
    }

    return 0;
}

```

- Lalu Compile dan jalankan dengan `gcc Bankers_Algoritma.c -o Bankers_Algoritma` dan `./Bankers_Algoritma`

Program itu ngelakuin Banker's Algorithm, yaitu algoritma buat ngecek apakah permintaan resource dari suatu proses aman buat sistem atau malah bikin sistem jadi deadlock.

Program punya data tentang berapa resource yang tersedia, berapa yang sudah dipakai proses, dan berapa kebutuhan maksimal tiap proses.

Saat kamu masukin nomor proses dan permintaan resource (misal: 1 0 2), program bakal ngecek beberapa hal:

Apakah permintaan melebihi kebutuhan maksimal proses?

Kalau iya langsung ditolak (kayak yang kejadian di proses nomor 3).

Apakah resource yang diminta tersedia?

Kalau kurang ditolak juga.

Kalau lolos dua cek di atas, program bakal simulasi:

“Kalau permintaan ini dikasih, sistem masih aman gak?”

Kalau simulasi menunjukkan urutan eksekusi proses masih mungkin sistem SAFE request disetujui.

Kalau bakal bikin deadlock UNSAFE request ditolak.

```

rafi@satelit: ~
❯ nano Bankers_Algoritma.c
rafi@satelit:~$ gcc Bankers_Algoritma.c -o Bankers_Algoritma
rafi@satelit:~$ ./Bankers_Algoritma
=== BANKER'S ALGORITHM ===
Proses yang meminta resource (0-3): 3
Masukkan request resource (3 angka, format: A B C): 1 0 2
ERROR: Request melebihi kebutuhan proses!
rafi@satelit:~$ ./Bankers_Algoritma
=== BANKER'S ALGORITHM ===
Proses yang meminta resource (0-3): 1
Masukkan request resource (3 angka, format: A B C): 1 0 2
STATUS: Sistem dalam keadaan SAFE. Request DISETUJUI.
rafi@satelit:~$

```

1.4 Tugas 4

1.4.1 Kasus Nyata (Real Case)

- Buat simulasi deadlock untuk kasus: "Dua orang ingin transfer uang antar rekening secara bersamaan"
 - Orang A: transfer dari Rekening 1 ke Rekening 2
 - Orang B: transfer dari Rekening 2 ke Rekening 1
 - Implementasikan deadlock prevention
- Buat File nano deadlock_transfer.c

Yang didalam file tersebut berisi Syntax :

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t rek1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t rek2 = PTHREAD_MUTEX_INITIALIZER;

int saldo1 = 1000;
int saldo2 = 1000;

// Orang A: transfer dari Rek 1 → Rek 2
void* orangA(void* arg) {
    printf("A: Kunci Rekening 1...\n");
    pthread_mutex_lock(&rek1);
    printf("A: Rekening 1 terkunci!\n");

    sleep(1);

    printf("A: Kunci Rekening 2...\n");
    pthread_mutex_lock(&rek2);
    printf("A: Rekening 2 terkunci!\n");

    saldo1 -= 100;
    saldo2 += 100;
    printf("A: Transfer berhasil\n");

    pthread_mutex_unlock(&rek2);
    pthread_mutex_unlock(&rek1);
    return NULL;
}

// Orang B: transfer dari Rek 2 → Rek 1
void* orangB(void* arg) {
    printf("B: Kunci Rekening 2...\n");
    pthread_mutex_lock(&rek2);
```

```

printf("B: Rekening 2 terkunci!\n");

sleep(1);

printf("B: Kunci Rekening 1...\n");
pthread_mutex_lock(&rek1);
printf("B: Rekening 1 terkunci!\n");

saldo2 -= 100;
saldo1 += 100;
printf("B: Transfer berhasil\n");

pthread_mutex_unlock(&rek1);
pthread_mutex_unlock(&rek2);
return NULL;
}

int main() {
    pthread_t t1, t2;

    printf("=== SIMULASI DEADLOCK TRANSFER UANG ===\n\n");

    pthread_create(&t1, NULL, orangA, NULL);
    pthread_create(&t2, NULL, orangB, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Program selesai (sebenarnya deadlock).\n");
    return 0;
}

```

- Lalu buat file nano no_deadlock_transfer.c

Yang didalam berisi syntax :

```

#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t rek1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t rek2 = PTHREAD_MUTEX_INITIALIZER;

int saldo1 = 1000;
int saldo2 = 1000;

// Fungsi helper: ambil kunci secara terurut
void lock_ordered(pthread_mutex_t* a, pthread_mutex_t* b) {
    if (a < b) {
        pthread_mutex_lock(a);

```

```

        pthread_mutex_lock(b);
    } else {
        pthread_mutex_lock(b);
        pthread_mutex_lock(a);
    }
}

void unlock_ordered(pthread_mutex_t* a, pthread_mutex_t* b) {
    if (a < b) {
        pthread_mutex_unlock(b);
        pthread_mutex_unlock(a);
    } else {
        pthread_mutex_unlock(a);
        pthread_mutex_unlock(b);
    }
}

// Transfer generik
void transfer(pthread_mutex_t* src, pthread_mutex_t* dst,
             int* s_saldo, int* d_saldo, int jumlah, char nama) {

    printf("%c: Mencoba transfer...\n", nama);

    lock_ordered(src, dst);
    printf("%c: Semua rekening terkunci!\n", nama);

    *s_saldo -= jumlah;
    *d_saldo += jumlah;

    printf("%c: Transfer %d berhasil!\n", nama, jumlah);

    unlock_ordered(src, dst);
}

// Orang A: Rek1 → Rek2
void* orangA(void* arg) {
    transfer(&rek1, &rek2, &saldo1, &saldo2, 100, 'A');
    return NULL;
}

// Orang B: Rek2 → Rek1
void* orangB(void* arg) {
    transfer(&rek2, &rek1, &saldo2, &saldo1, 100, 'B');
    return NULL;
}

int main() {
    pthread_t t1, t2;

```



```

    printf("=== DEADLOCK PREVENTION: LOCK ORDERING
    ===\n\n");

    pthread_create(&t1, NULL, orangA, NULL);
    pthread_create(&t2, NULL, orangB, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\nSaldo akhir: Rek1 = %d, Rek2 = %d\n", saldo1, saldo2);
    printf("Program selesai tanpa deadlock!\n");
    return 0;
}

```

- Lalu compile dan jalankan yang pertama dengan gcc deadlock_transfer.c -o deadlock_transfer -lpthread dan ./deadlock_transfer, seharusnya akan terjaid HANG pada file yang pertama maka langsung keluar menggunakan ctrl + c
- Lalu setelah itu compile dan jalankan file yang kedua dengan gcc no_deadlock_transfer.c -o no_deadlock_transfer -lpthread dan ./no_deadlock_transfer maka hasil nya akan seperti ini ini

Program yang kamu jalankan itu sebenarnya nunjukkin perbedaan antara kode yang bisa kena deadlock dan kode yang sudah diperbaiki biar nggak deadlock.

1. Program pertama: deadlock_transfer.c

Makna:

Ini contoh transfer uang antara dua rekening, tapi tiap thread ngambil kunci (mutex) dengan urutan yang beda.

Thread A ngunci Rekening 1 dulu → baru Rekening 2

Thread B ngunci Rekening 2 dulu → baru Rekening 1

Karena order-nya beda, dua thread bisa saling nunggu mutex yang lagi dipegang thread lain.

Hasilnya? Deadlock, program nge-freeze dan gak lanjut.

Inti pesan program:

Ini simulasi gimana deadlock bisa kejadian kalau dua thread ngunci resource secara silang.

2. Program kedua: no_deadlock_transfer.c

Makna:

Ini versi yang sudah diperbaiki pakai teknik deadlock prevention dengan lock ordering.

Semua thread DIWAJIBKAN ngambil kunci rekening dengan urutan yang sama
(misal: selalu kunci rekening bernomor kecil dulu, baru rekening besar)

Dengan begitu:

Gak ada thread yang ngambil kunci secara silang

Gak ada kondisi saling tunggu

Transfer bisa jalan aman tanpa deadlock

Makanya output-nya menunjukkan:

Semua rekening sukses dikunci

Transfer jalan

Program selesai normal

Inti pesan program:

Kalau order pengambilan lock diatur konsisten, deadlock bisa dihindari.

Program ini sebenarnya lagi ngajarin konsep penting di pemrograman multithreading:

Deadlock bisa muncul kalau dua (atau lebih) thread ngambil resource dengan urutan berbeda. Solusinya: pakai lock ordering biar semua thread ngunci resource dengan urutan sama.

```
rafi@satelit: ~  
rafi@satelit:~$ nano deadlock_transfer.c  
rafi@satelit:~$ gcc deadlock_transfer.c -o deadlock_transfer -lpthread  
rafi@satelit:~$ ./deadlock_transfer  
=== SIMULASI DEADLOCK TRANSFER UANG ===  
  
A: Kunci Rekening 1...  
A: Rekening 1 terkunci!  
B: Kunci Rekening 2...  
B: Rekening 2 terkunci!  
A: Kunci Rekening 2...  
B: Kunci Rekening 1...  
^C  
rafi@satelit:~$ nano no_deadlock_transfer.c  
rafi@satelit:~$ gcc no_deadlock_transfer.c -o no_deadlock_transfer -lpthread  
rafi@satelit:~$ ./no_deadlock_transfer  
=== DEADLOCK PREVENTION: LOCK ORDERING ===  
  
A: Mencoba transfer...  
A: Semua rekening terkunci!  
A: Transfer 100 berhasil!  
B: Mencoba transfer...  
B: Semua rekening terkunci!  
B: Transfer 100 berhasil!  
  
Saldo akhir: Rek1 = 1000, Rek2 = 1000  
Program selesai tanpa deadlock!
```